IDENTIFICAÇÃO DAS FONTES DE DADOS

A base de dados utilizada neste projeto representa registros operacionais da empresa Cannoli, uma rede de lojas do setor alimentício. Os dados incluem informações relacionadas a clientes, pedidos realizados, campanhas de marketing, lojas e templates de comunicação.

As informações estão distribuídas em múltiplos arquivos no formato CSV, representando diferentes entidades do sistema, como clientes, lojas, pedidos e campanhas. Cada arquivo corresponde a uma tabela distinta dentro do modelo de dados utilizado na análise.

As principais tabelas presentes na base são: CUSTOMER, CUSTOMERADDRESS, STORE, STOREORDER, CAMPAIGN, CAMPAIGNxORDER e TEMPLATE, cada uma armazenando informações específicas sobre as operações da empresa.

A utilização dessas tabelas permite analisar diferentes aspectos do negócio, como o comportamento de compra dos clientes, o desempenho das lojas, o impacto das campanhas de marketing e os padrões de vendas ao longo do tempo.

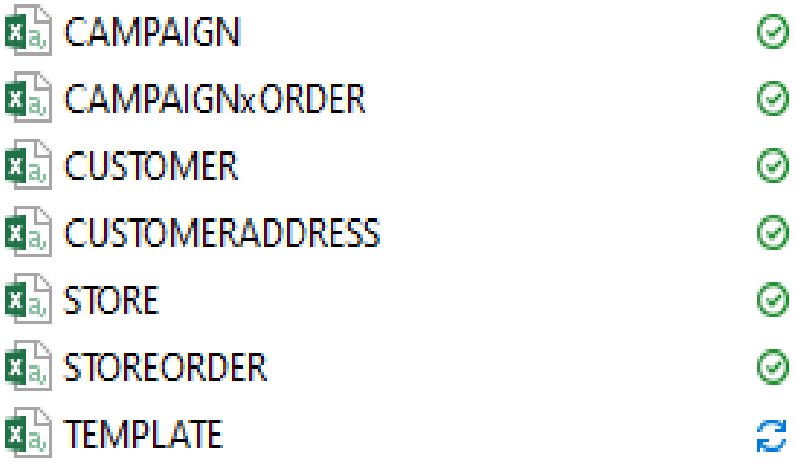

Figura 1 – Arquivos da Base de dados da Cannoli

COLETA DOS DADOS

Para a coleta, manipulação e visualização dos dados foi utilizada a linguagem de programação Python, no ambiente Google Colab, juntamente com bibliotecas amplamente utilizadas em análise de dados.

As principais bibliotecas utilizadas foram:

- Pandas: utilizada para leitura, manipulação e análise dos dados, permitindo trabalhar com estruturas de dados do tipo DataFrame.

- Matplotlib: biblioteca utilizada para criação de gráficos e visualizações básicas dos dados.

- Seaborn: biblioteca baseada em Matplotlib que facilita a criação de gráficos estatísticos com visual mais elaborado.

DESCRIÇÃO DOS DADOS

Após a etapa de coleta dos dados, foi realizada uma análise inicial com o objetivo de compreender a estrutura das tabelas, os tipos de dados presentes e algumas características gerais da base. Essa etapa é importante para garantir que os dados foram carregados corretamente e para identificar possíveis padrões ou inconsistências antes do início das análises exploratórias.

Para essa verificação inicial foram utilizadas funções da biblioteca Pandas, como head(), info() e describe(). Essas funções permitem visualizar os primeiros registros das tabelas, identificar os tipos de dados presentes em cada coluna e obter estatísticas descritivas das variáveis numéricas.

Visualização Inicial dos Dados

Inicialmente foi utilizada a função head() para visualizar os primeiros registros das tabelas. Essa função retorna as primeiras linhas do conjunto de dados, permitindo uma inspeção rápida da estrutura das informações armazenadas em cada tabela.

Essa etapa possibilita verificar se os dados foram carregados corretamente, além de permitir uma primeira compreensão das variáveis presentes na base e do tipo de informação armazenada em cada coluna.

index	id	storeid	customerid	scheduledat	totalamount	subtotalamount	discountamount	taxamount	saleschannel	status	ordertype	createdat	createdby
0	6986251f-3cda-4671-acb9-8bc9646ed2a9	0d756ccf-7f47-4d8d-a2fb-aa78226a401d	3d04ea93-8ed5-4d29-b641-1a9a41e8820f	2025-08-26 20:54:25.674-03	59.9	69.9	10.0	0.0	IFOOD	16	DELIVERY	2025-08-26 20:54:25.674-03	Cannoli Hangfire
1	d9b8dc9a-d4da-4a8b-a57a-da6813074818	4bb014bb-8b1d-489a-aded-2060f3fda27e	5679bc63-7f77-450c-b089-423cf0c564c9	2025-11-29 12:51:14.187-03	114.0	114.0	0.0	0.0	CARDAPIOWEB	16	INDOOR	2025-11-29 12:51:14.187-03	Cannoli API
2	0b22d312-5271-4c2b-b75e-2579fadc759f	4bb014bb-8b1d-489a-aded-2060f3fda27e	5679bc63-7f77-450c-b089-423cf0c564c9	2025-11-14 20:52:45.347-03	83.9	83.9	0.0	0.0	CARDAPIOWEB	16	INDOOR	2025-11-14 20:52:45.347-03	Cannoli API
3	8782daec-d43b-4b23-9610-3797156391f4	0c02619d-0c57-492d-9660-dd0f357a2f0c	19cc7cf9-1dad-4f5d-8c28-e65f1fd212de	2025-11-01 10:39:32.516-03	45.54	44.55	0.0	0.99	IFOOD	16	DELIVERY	2025-11-01 10:39:32.516-03	Cannoli API
4	e4c15a26-d0ca-4a18-af1d-ae430e27cc25	0c02619d-0c57-492d-9660-dd0f357a2f0c	9b2b5739-1e9f-42cb-bddb-cd4f150ec9e2	2025-11-14 21:21:44.201-03	49.99	59.0	10.0	0.99	IFOOD	16	DELIVERY	2025-11-14 21:21:44.201-03	Cannoli API

Figura 2 - Visualização inicial da tabela STOREORDER utilizando a função head().

index	id	name	gender	dateofbirth	status	isenriched	enrichedat	enrichedby	createdat	updatedat	updatedby	birthmonth	birthday
0	f97b1646-19b7-48af-b134-c0ae196e8297	JAIME	M	1987-03-05	1	true	2025-09-02 19:12:35.009-03	Cannoli Food	2025-07-03 19:19:02.159-03	2025-09-02 19:13:00.346-03	Admin	3.0	5.0
1	1fa651fd-b539-49c8-8ea7-e714a705f946	Miche	NaN	NaN	1	NaN	NaN	NaN	2025-10-23 21:29:06.931-03	2025-10-23 21:29:06.931-03	Cannoli API	NaN	NaN
2	85fc4a88-b5e8-470a-9ed7-33643e2be3eb	Kéren	N	NaN	1	NaN	NaN	NaN	2025-08-22 11:46:31.006-03	2025-09-16 16:32:10.387-03	Paulo Bertelli	NaN	NaN
3	7356b8f0-8973-4982-935b-0d7e10d1512e	Rodri	N	NaN	1	NaN	NaN	NaN	2025-09-16 16:32:13.32-03	2025-09-16 16:32:13.32-03	Paulo Bertelli	NaN	NaN
4	daba3023-f64f-4862-90af-bcd6481c9fac	Júlia	O	1979-10-03	1	NaN	NaN	NaN	2025-07-04 19:31:20.059-03	2025-09-13 04:48:09.044-03	HorusBI App	10.0	3.0

Figura 3 - Visualização inicial da tabela CUSTOMER utilizando a função head().

Estrutura das Tabelas

Em seguida, foi utilizada a função info() para analisar a estrutura das tabelas. Essa função fornece informações importantes sobre o conjunto de dados, como o número total de registros, a quantidade de colunas presentes na tabela, os tipos de dados de cada variável e a presença de valores nulos.

A análise da estrutura dos dados é fundamental para compreender como as informações estão organizadas e para identificar possíveis necessidades de tratamento ou padronização dos dados antes da realização das análises.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 118936 entries, 0 to 118935
Data columns (total 13 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   id              118936 non-null  object
 1   storeid         118936 non-null  object
 2   customerid      118936 non-null  object
 3   scheduledat     118936 non-null  object
 4   totalamount     118936 non-null  float64
 5   subtotalamount  118936 non-null  float64
 6   discountamount  118936 non-null  float64
 7   taxamount       118936 non-null  float64
 8   saleschannel    118936 non-null  object
 9   status          118936 non-null  int64
 10  ordertype       118936 non-null  object
 11  createdat       118936 non-null  object
 12  createdby       118936 non-null  object
dtypes: float64(4), int64(1), object(8)
memory usage: 11.8+ MB
```

Figura 4 - Estrutura da tabela STOREORDER obtida com a função info().

Estatísticas Descritivas

Para aprofundar a compreensão dos dados numéricos presentes na base, foi utilizada a função describe(). Essa função fornece um resumo estatístico das variáveis numéricas, incluindo métricas como média, desvio padrão, valores mínimos e máximos, além de quartis da distribuição.

Essas estatísticas ajudam a identificar padrões nos dados, compreender a distribuição dos valores e detectar possíveis valores extremos que possam influenciar as análises posteriores.

index	status	birthmonth	birthday
count	160710.0	8903.0	8903.0
mean	1.0	6.840727844546782	16.83151746602269
std	0.0	4.7088466300611005	12.219962583053862
min	1.0	1.0	1.0
25%	1.0	1.0	2.0
50%	1.0	7.0	20.0
75%	1.0	12.0	30.0
max	1.0	12.0	31.0

Figura 5 - Estatísticas descritivas da tabela STOREORDER obtidas com a função describe().

Com essas análises iniciais foi possível obter uma visão geral da estrutura da base de dados e das principais características das informações disponíveis. Essa etapa constitui um passo fundamental para orientar as análises exploratórias e a construção dos indicadores que serão apresentados no dashboard.

EXPLORAÇÃO DOS DADOS

Após a análise inicial da estrutura da base de dados, foi realizada uma etapa de exploração com o objetivo de identificar padrões, tendências e relações relevantes entre as variáveis presentes nas tabelas.

A exploração dos dados foi realizada no ambiente Google Colab, utilizando as bibliotecas Matplotlib e Seaborn para a construção de gráficos e visualizações estatísticas. Essas visualizações auxiliam na interpretação dos dados e permitem identificar comportamentos importantes relacionados às vendas, ao perfil dos clientes e ao desempenho das campanhas de marketing.

A partir dessa análise exploratória foram geradas visualizações que ajudam a compreender melhor o comportamento dos dados e identificar possíveis indicadores relevantes para o desenvolvimento do dashboard.

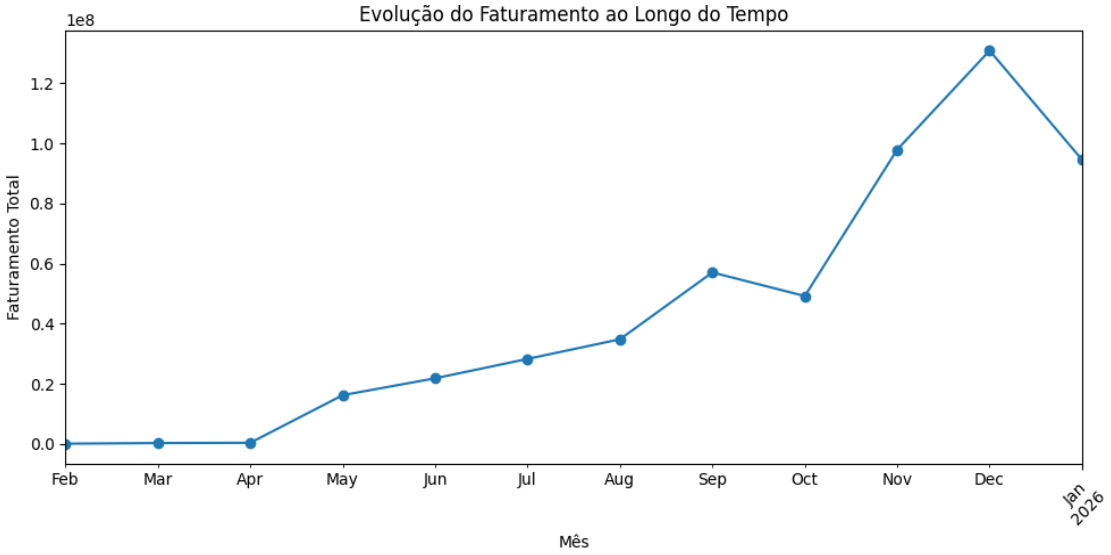

Figura 6 — Evolução do faturamento ao longo do tempo.

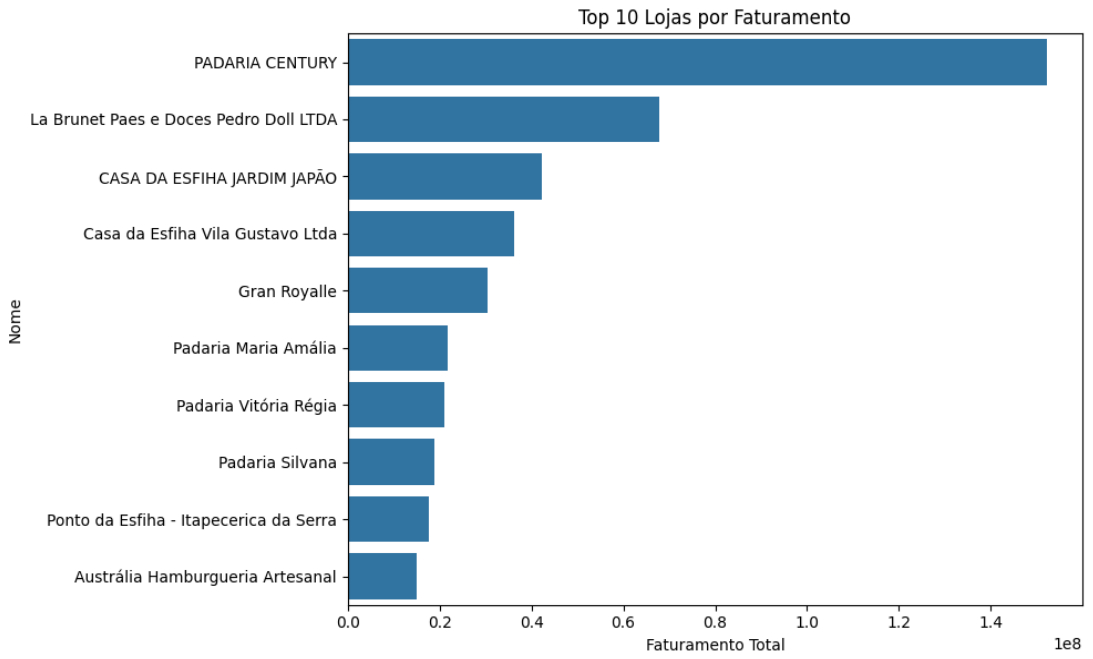

Figura 7 — Lojas com maior volume de faturamento.

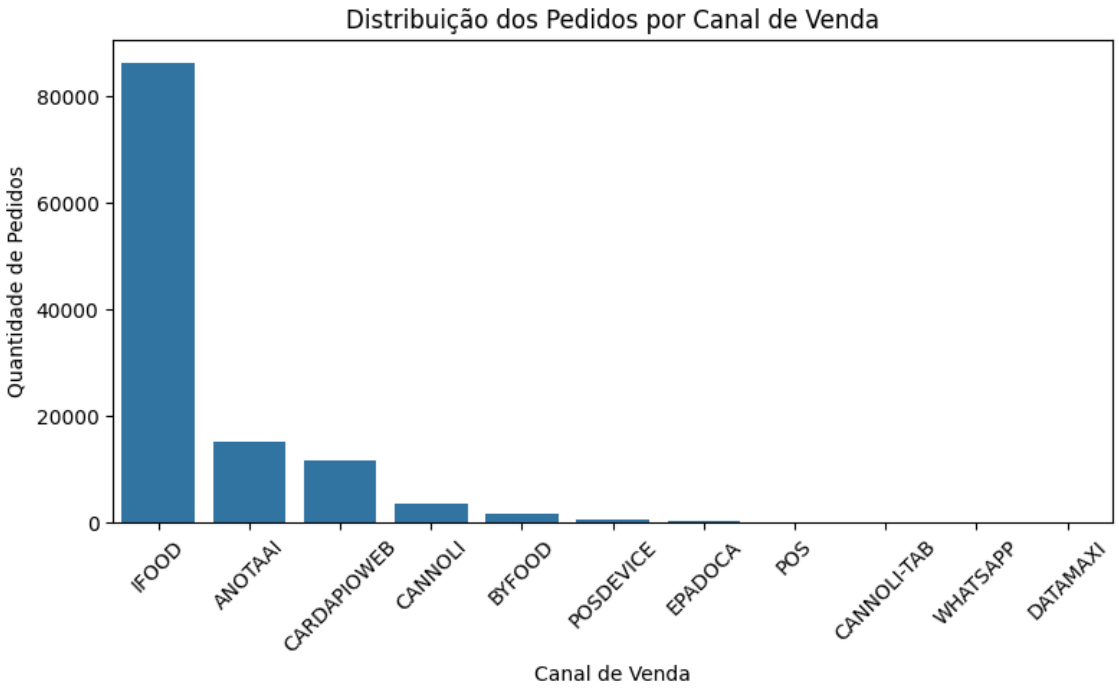

Figura 8 — Distribuição dos pedidos por canal de venda.

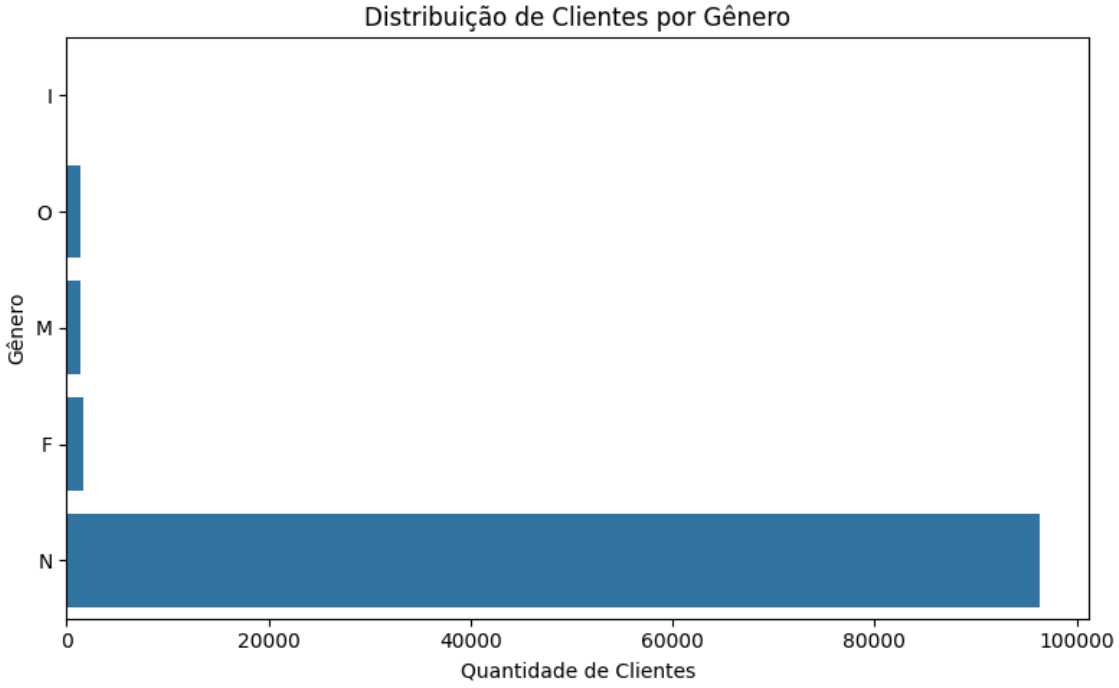

Figura 9 — Distribuição de clientes por gênero.

A análise exploratória permitiu identificar padrões importantes nos dados, como a concentração de faturamento em determinadas lojas, diferenças na participação dos canais de venda e características relacionadas ao perfil dos clientes. Essas informações servem como base para a construção dos indicadores que serão utilizados no dashboard.

SELEÇÃO DOS DADOS

Após a análise exploratória inicial, foram selecionadas as tabelas **STOREORDER, STORE, CUSTOMER, CAMPAIGN e CAMPAIGNORDER**, por apresentarem maior relevância para a construção do dashboard.

Foram priorizadas colunas relacionadas a:

- identificação (id, storeid, customerid)
- valores financeiros (totalamount, subtotalamount)
- datas (scheduledat, createdat)
- categorização (saleschannel, ordertype, status)

```
store_order_sel = storeOrder[
    ['id', 'storeid', 'customerid', 'scheduledat',
     'totalamount', 'subtotalamount',
     'saleschannel', 'ordertype', 'status']
].copy()
```

Figura 10 — Exemplo de seleção de dados no ambiente Python

Essa etapa garante que apenas dados relevantes sejam utilizados, reduzindo complexidade e melhorando a performance das análises.

LIMPEZA E PADRONIZAÇÃO DOS DADOS

Nesta etapa foram tratadas inconsistências encontradas nos dados, como:

- valores nulos
- tipos de dados incorretos
- registros duplicados

Conversão de colunas para tipo numérico:

```
store_order_sel['totalamount'] = pd.to_numeric(store_order_sel['totalamount'], errors='coerce')

store_order_sel['subtotalamount'] = pd.to_numeric(store_order_sel['subtotalamount'], errors='coerce')
```

Conversão de Datas:

```
store_order_sel['scheduledat'] = pd.to_datetime(store_order_sel['scheduledat'], errors='coerce')
```

Remoção de Duplicatas:

```
store_order_sel = store_order_sel.drop_duplicates()
```

Essa etapa foi essencial para garantir a confiabilidade dos resultados.

DERIVAÇÃO DOS DADOS

Foram criadas novas variáveis a partir das existentes para facilitar análises temporais e segmentações.

```
store_order_sel['ano'] = store_order_sel['scheduledat'].dt.year
store_order_sel['mes'] = store_order_sel['scheduledat'].dt.month
store_order_sel['dia'] = store_order_sel['scheduledat'].dt.day
```

Figura 11 — Extração de informações de data

Essas variáveis permitem análises como:

- faturamento mensal
- evolução ao longo do tempo
- comportamento por período

INTEGRAÇÃO DOS DADOS

As tabelas foram integradas utilizando chaves comuns, permitindo análises mais completas.

```
store_order_loja = store_order_sel.merge(
    store,
    left_on='storeid',
    right_on='id',
    how='left'
)
```

Figura 12 — Exemplo de integração entre pedidos e lojas

Também foi possível integrar dados de clientes e campanhas, permitindo cruzamentos como:

- desempenho por loja
- comportamento por cliente
- impacto de campanhas nos pedidos

FORMATAÇÃO DOS DADOS

Após a integração, os dados foram agregados para gerar indicadores utilizados no dashboard.

```
faturamento_mensal = (
    store_order_sel
    .dropna(subset=['scheduledat'])
    .groupby(store_order_sel['scheduledat'].dt.to_period('M'))['totalamount']
    .sum()
)
```

Figura 13 — Exemplo de faturamento por mês

```
vendas_canal = (
    store_order_sel
    .groupby('saleschannel')
    .agg(
        total_vendas=('totalamount', 'sum'),
        quantidade=('id', 'count')
    )
    .sort_values(by='total_vendas', ascending=False)
)
```

Figura 14 — Exemplo de análise por canal de venda:

```
tipo_pedido = store_order_sel['ordertype'].value_counts(normalize=True) * 100
```

Figura 15 — Exemplo de distribuição por tipo de pedido

Essas transformações são fundamentais para a construção dos gráficos do dashboard, como:

- faturamento ao longo do tempo
- distribuição por canal de venda
- análise por tipo de pedido